DBMS Practical Question Bank

Solutions for Questions 1–20 (A and B)

MySQL Command Line and MongoDB Shell (mongosh)

Prepared as a practical-exam revision document with copy-ready commands, sample data, queries, expected results, and viva questions where available.

Important environment note: Run MySQL solutions in MySQL Command Line and MongoDB solutions in `mongosh`. Oracle PL/SQL-specific exercises were adapted to MySQL equivalents in the solutions; their syntax is not identical to Oracle PL/SQL.

Question 1(A) — Student Information System

Environment: MySQL Command Line

Create a student table, insert records, query marks and departments, calculate aggregates, update a record, and delete a record.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS STUDENT;
CREATE TABLE STUDENT (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Department VARCHAR(30) NOT NULL,
    `Year` INT,
    City VARCHAR(40),
    Marks DECIMAL(5,2)
);

INSERT INTO STUDENT VALUES
(1, 'Amit', 'IT', 2, 'Pune', 82),
(2, 'Priya', 'CS', 2, 'Mumbai', 91),
(3, 'Rahul', 'IT', 2, 'Pune', 68),
(4, 'Sneha', 'ENTC', 2, 'Nashik', 77),
(5, 'Vijay', 'IT', 2, 'Pune', 88),
(6, 'Neha', 'CS', 2, 'Delhi', 73),
(7, 'Karan', 'ENTC', 2, 'Mumbai', 80),
(8, 'Rohan', 'IT', 2, 'Pune', 59);

SELECT * FROM STUDENT;
SELECT * FROM STUDENT WHERE Marks > 75;
SELECT * FROM STUDENT WHERE Department = 'IT';
SELECT MAX(Marks) AS HighestMarks FROM STUDENT;
SELECT MIN(Marks) AS LowestMarks FROM STUDENT;
SELECT AVG(Marks) AS AverageMarks FROM STUDENT;
SELECT Department, AVG(Marks) AS AverageMarks
FROM STUDENT GROUP BY Department;

UPDATE STUDENT SET Marks = 90 WHERE RollNo = 3;
DELETE FROM STUDENT WHERE RollNo = 8;
SELECT * FROM STUDENT;
```

Expected results / notes

• Use `WHERE` to filter rows and aggregate functions such as `MAX`, `MIN`, and `AVG` to summarize marks.

Viva questions

What is a primary key? A column or combination of columns that uniquely identifies each row.

What does `GROUP BY` do? It groups rows so aggregate functions can be calculated for each group.

Question 1(B) — Stored Procedure to Calculate Grade

Environment: MySQL Command Line

Create a stored procedure that assigns a grade according to the student's marks.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS STUD_MARKS;
CREATE TABLE STUD_MARKS (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

INSERT INTO STUD_MARKS VALUES
(1, 'Amit', 95), (2, 'Priya', 82), (3, 'Rahul', 67),
(4, 'Sneha', 45), (5, 'Vijay', 32);

DROP PROCEDURE IF EXISTS PROC_GRADE;
DELIMITER //
```

```
CREATE PROCEDURE PROC_GRADE()
BEGIN
    SELECT RollNo, Name, Marks,
        CASE
            WHEN Marks >= 90 THEN 'Distinction'
            WHEN Marks >= 75 THEN 'First Class'
            WHEN Marks >= 60 THEN 'Second Class'
            WHEN Marks >= 40 THEN 'Pass'
            ELSE 'Fail'
        END AS Grade
    FROM STUD_MARKS;
END //
DELIMITER ;

CALL PROC_GRADE();
```

Expected results / notes

• The procedure returns each student's marks and calculated grade.

Viva questions

What is a stored procedure? A named group of SQL statements stored in the database.

What does `CASE` do? It returns a result based on conditional branches.

Question 2(A) — Employee Management System

Environment: MySQL Command Line

Create an employee table and run salary, sorting, department-count, and maximum-salary queries.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Department VARCHAR(30),
    Salary DECIMAL(10,2)
);

INSERT INTO EMPLOYEE VALUES
(1, 'Amit', 'IT', 60000),
(2, 'Priya', 'HR', 45000),
(3, 'Rahul', 'IT', 75000),
(4, 'Sneha', 'Finance', 52000),
(5, 'Vijay', 'HR', 48000),
(6, 'Neha', 'IT', 68000),
(7, 'Karan', 'Finance', 57000),
(8, 'Rohan', 'Sales', 42000);

SELECT * FROM EMPLOYEE WHERE Salary > 50000;
SELECT * FROM EMPLOYEE ORDER BY Salary DESC;
SELECT Department, COUNT(*) AS EmployeeCount
FROM EMPLOYEE GROUP BY Department;
SELECT MAX(Salary) AS MaximumSalary FROM EMPLOYEE;
```

Expected results / notes

• `ORDER BY Salary DESC` sorts salaries from highest to lowest.

Viva questions

What does `COUNT(*)` do? Counts the rows in each group.

What does `DESC` mean? Descending order.

Question 2(B) — MongoDB Products Queries

Environment: MongoDB Shell (mongosh)

Create a products collection, insert records, filter products by price, sort, limit, and project selected fields.

Complete code

```
use DBMS_Practical

db.products.drop()
db.products.insertMany([
  { productId: 1, name: "Laptop", category: "Electronics", price: 55000 },
  { productId: 2, name: "Mouse", category: "Electronics", price: 800 },
  { productId: 3, name: "Keyboard", category: "Electronics", price: 1500 },
  { productId: 4, name: "Chair", category: "Furniture", price: 3500 },
  { productId: 5, name: "Desk", category: "Furniture", price: 7000 },
  { productId: 6, name: "Phone", category: "Electronics", price: 22000 },
  { productId: 7, name: "Lamp", category: "Home", price: 1200 },
  { productId: 8, name: "Tablet", category: "Electronics", price: 18000 }
])

db.products.find({ price: { $gt: 1000 } })
db.products.find().sort({ price: -1 }).limit(3)
db.products.find({}, { _id: 0, name: 1, price: 1 })
```

Expected results / notes

• `$gt` means greater than; `.limit(3)` returns at most three documents; projection selects the `name` and `price` fields.

Viva questions

What is a collection? A group of MongoDB documents.

What does projection do? It specifies which fields to include or exclude in returned documents.

Question 3(A) — Book Table Queries

Environment: MySQL Command Line

Create a book table, insert records, sort by category, filter prices, aggregate by category, find maximum price, and update copies.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS BOOK;
CREATE TABLE BOOK (
    BookID INT PRIMARY KEY,
    Title VARCHAR(100),
    Author VARCHAR(50),
    Category VARCHAR(40),
    Price DECIMAL(8,2),
    Copies INT
);

INSERT INTO BOOK VALUES
(1, 'DBMS Basics', 'Amit', 'Education', 450, 10),
(2, 'Java Programming', 'Priya', 'Programming', 650, 8),
(3, 'Python Guide', 'Rahul', 'Programming', 550, 12),
(4, 'Web Development', 'Sneha', 'Programming', 700, 5),
(5, 'Database Design', 'Vijay', 'Education', 500, 7),
(6, 'Networks', 'Neha', 'Education', 600, 9),
(7, 'C Programming', 'Karan', 'Programming', 400, 11),
(8, 'Algorithms', 'Rohan', 'Education', 800, 4);

SELECT * FROM BOOK ORDER BY Category;
SELECT * FROM BOOK WHERE Price > 500;
SELECT Category, COUNT(*) AS BookCount FROM BOOK GROUP BY Category;
SELECT MAX(Price) AS HighestPrice FROM BOOK;
UPDATE BOOK SET Copies = Copies + 2 WHERE BookID = 1;
SELECT * FROM BOOK;
```

Expected results / notes

• The update increases the number of copies of book 1 by two.

Viva questions

What does `UPDATE` do? Modifies existing records.

What does `MAX()` return? The maximum value in a selected set of rows.

Question 3(B) — Audit Triggers

Environment: MySQL Command Line

Create a library table and an audit table. Triggers record old values before update and delete operations.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TRIGGER IF EXISTS library_before_update;
DROP TRIGGER IF EXISTS library_before_delete;
DROP TABLE IF EXISTS LIBRARY_AUDIT;
DROP TABLE IF EXISTS LIBRARY;

CREATE TABLE LIBRARY (
    BookID INT PRIMARY KEY,
    Title VARCHAR(100),
    Author VARCHAR(50),
    Copies INT
);

CREATE TABLE LIBRARY_AUDIT (
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    BookID INT,
    Title VARCHAR(100),
    Author VARCHAR(50),
    Copies INT,
    ActionType VARCHAR(20),
    ActionTime DATETIME
);

INSERT INTO LIBRARY VALUES
(1, 'DBMS Basics', 'Amit', 10),
(2, 'Java Programming', 'Priya', 8);

DELIMITER //
CREATE TRIGGER library_before_update
BEFORE UPDATE ON LIBRARY
FOR EACH ROW
BEGIN
    INSERT INTO LIBRARY_AUDIT
    (BookID, Title, Author, Copies, ActionType, ActionTime)
    VALUES
    (OLD.BookID, OLD.Title, OLD.Author, OLD.Copies, 'UPDATE', NOW());
END //

CREATE TRIGGER library_before_delete
BEFORE DELETE ON LIBRARY
FOR EACH ROW
BEGIN
    INSERT INTO LIBRARY_AUDIT
    (BookID, Title, Author, Copies, ActionType, ActionTime)
    VALUES
    (OLD.BookID, OLD.Title, OLD.Author, OLD.Copies, 'DELETE', NOW());
END //
DELIMITER ;

UPDATE LIBRARY SET Copies = 15 WHERE BookID = 1;
DELETE FROM LIBRARY WHERE BookID = 2;
SELECT * FROM LIBRARY_AUDIT;
```

Expected results / notes

• `OLD` refers to the record values before the triggering update or delete.

Viva questions

What is a trigger? A database program automatically executed in response to an event.

What is `OLD`? It refers to the old row values in an applicable trigger.

Question 4(A) — Department and Employee JOINs

Environment: MySQL Command Line

Demonstrate INNER, LEFT, RIGHT, and FULL OUTER JOIN behavior. MySQL has no direct FULL OUTER JOIN syntax, so use LEFT JOIN UNION RIGHT JOIN.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS EMPLOYEE;
DROP TABLE IF EXISTS DEPARTMENT;

CREATE TABLE DEPARTMENT (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(40)
);

CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT
);

INSERT INTO DEPARTMENT VALUES (10, 'IT'), (20, 'HR'), (30, 'Finance');
INSERT INTO EMPLOYEE VALUES (1, 'Amit', 10), (2, 'Priya', 20), (3, 'Rahul', NULL);

-- INNER JOIN
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E INNER JOIN DEPARTMENT D ON E.DeptID = D.DeptID;

-- LEFT JOIN
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E LEFT JOIN DEPARTMENT D ON E.DeptID = D.DeptID;

-- RIGHT JOIN
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E RIGHT JOIN DEPARTMENT D ON E.DeptID = D.DeptID;

-- FULL OUTER JOIN equivalent in MySQL
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E LEFT JOIN DEPARTMENT D ON E.DeptID = D.DeptID
UNION
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E RIGHT JOIN DEPARTMENT D ON E.DeptID = D.DeptID;
```

Expected results / notes

• The `UNION` combines the results and removes duplicate rows.

Viva questions

What is INNER JOIN? Returns rows with matching values in both tables.

Does MySQL support FULL OUTER JOIN directly? No; a LEFT JOIN and RIGHT JOIN combined with UNION can emulate it.

Question 4(B) — Roll Call Migration with Cursor

Environment: MySQL Command Line

Insert roll-call records from an old table into a new table only when the roll number does not already exist.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS migrate_rollcall;
DROP TABLE IF EXISTS N_ROLLCALL;
DROP TABLE IF EXISTS O_ROLLCALL;

CREATE TABLE O_ROLLCALL (RollNo INT PRIMARY KEY, Name VARCHAR(50));
CREATE TABLE N_ROLLCALL (RollNo INT PRIMARY KEY, Name VARCHAR(50));

INSERT INTO O_ROLLCALL VALUES
(1, 'Amit'), (2, 'Priya'), (3, 'Rahul'), (4, 'Sneha');
INSERT INTO N_ROLLCALL VALUES (2, 'Priya');

DELIMITER //
CREATE PROCEDURE migrate_rollcall()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_RollNo INT;
    DECLARE v_Name VARCHAR(50);
    DECLARE inserted_count INT DEFAULT 0;
    DECLARE cur CURSOR FOR SELECT RollNo, Name FROM O_ROLLCALL;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_RollNo, v_Name;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        IF NOT EXISTS (SELECT 1 FROM N_ROLLCALL WHERE RollNo = v_RollNo) THEN
            INSERT INTO N_ROLLCALL VALUES (v_RollNo, v_Name);
            SET inserted_count = inserted_count + 1;
        END IF;
    END LOOP;
    CLOSE cur;

    SELECT inserted_count AS InsertedRecords;
END //
DELIMITER ;

CALL migrate_rollcall();
SELECT * FROM N_ROLLCALL;
```

Expected results / notes

• Existing roll numbers are skipped; only missing records are inserted.

Viva questions

What is a cursor? A database object used to process query results row by row.

What does a cursor handler do? It handles conditions such as reaching the end of the cursor result set.

Question 5(A) — Student Course Registration

Environment: MySQL Command Line

Create STUDENT, COURSE, and REGISTRATION tables; demonstrate joins, course filtering, students without registrations, and registration counts.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS REGISTRATION;
DROP TABLE IF EXISTS COURSE;
DROP TABLE IF EXISTS STUDENT;

CREATE TABLE STUDENT (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50)
);
CREATE TABLE COURSE (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50)
);
CREATE TABLE REGISTRATION (
    RollNo INT,
    CourseID INT,
    PRIMARY KEY (RollNo, CourseID),
    FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo),
    FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);

INSERT INTO STUDENT VALUES (1,'Amit'),(2,'Priya'),(3,'Rahul'),(4,'Sneha');
INSERT INTO COURSE VALUES (101,'DBMS'),(102,'Java'),(103,'Networks');
INSERT INTO REGISTRATION VALUES (1,101),(1,102),(2,101),(3,103);

-- Students and registered courses
SELECT S.Name, C.CourseName
FROM STUDENT S
JOIN REGISTRATION R ON S.RollNo = R.RollNo
JOIN COURSE C ON R.CourseID = C.CourseID;

-- Students registered for DBMS
SELECT S.Name
FROM STUDENT S
JOIN REGISTRATION R ON S.RollNo = R.RollNo
JOIN COURSE C ON R.CourseID = C.CourseID
WHERE C.CourseName = 'DBMS';

-- Students not registered for any course
```

```
SELECT S.RollNo, S.Name
FROM STUDENT S
WHERE NOT EXISTS (
    SELECT 1 FROM REGISTRATION R WHERE R.RollNo = S.RollNo
);

-- Number of students registered per course
SELECT C.CourseName, COUNT(R.RollNo) AS RegistrationCount
FROM COURSE C LEFT JOIN REGISTRATION R ON C.CourseID = R.CourseID
GROUP BY C.CourseID, C.CourseName;
```

Expected results / notes

• LEFT JOIN retains courses with zero registrations.

Viva questions

What is a foreign key? A reference to a key in another table.

Why use NOT EXISTS? To find rows for which no matching row exists in a related table.

Question 5(B) — MongoDB College Students CRUD

Environment: MongoDB Shell (mongosh)

Insert students, find marks above 75, update Rahul, and delete Rohan.

Complete code

```
use CollegeDB

db.students.drop()
db.students.insertMany([
  { rollNo: 1, name: "Shivraj", marks: 82 },
  { rollNo: 2, name: "Shreyansh", marks: 78 },
  { rollNo: 3, name: "Satyajeet", marks: 88 },
  { rollNo: 4, name: "Rahul", marks: 70 },
  { rollNo: 5, name: "Aditya", marks: 91 },
  { rollNo: 6, name: "Vivek", marks: 74 },
  { rollNo: 7, name: "Akash", marks: 80 },
  { rollNo: 8, name: "Rohan", marks: 68 }
])

db.students.find({ marks: { $gt: 75 } })
db.students.updateOne({ name: "Rahul" }, { $set: { marks: 76 } })
db.students.deleteOne({ name: "Rohan" })
db.students.find()
```

Expected results / notes

• This exercise uses the `CollegeDB` database as in the practical example.

Viva questions

What is `find()`? Retrieves documents matching a filter.

What does `$set` do? Sets a field to a specified value.

Question 6(A) — Doctor and Patient Database

Environment: MySQL Command Line

Use joins to list patients and doctors, find patients older than 60, count patients per doctor, and find doctors with no patients.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS PATIENT;
DROP TABLE IF EXISTS DOCTOR;

CREATE TABLE DOCTOR (
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(50),
    Specialization VARCHAR(50)
);
CREATE TABLE PATIENT (
```

```
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(50),
    Age INT,
    DoctorID INT,
    FOREIGN KEY (DoctorID) REFERENCES DOCTOR(DoctorID)
);

INSERT INTO DOCTOR VALUES
(1,'Dr. Sharma','General'),(2,'Dr. Patil','Cardiology'),
(3,'Dr. Joshi','Neurology'),(4,'Dr. Mehta','Pediatrics');

INSERT INTO PATIENT VALUES
(101,'Amit',65,1),(102,'Priya',45,2),(103,'Rahul',72,1),
(104,'Sneha',30,3),(105,'Vijay',61,2);

SELECT P.PatientName, P.Age, D.DoctorName
FROM PATIENT P JOIN DOCTOR D ON P.DoctorID = D.DoctorID;

SELECT * FROM PATIENT WHERE Age > 60;

SELECT D.DoctorName, COUNT(P.PatientID) AS PatientCount
FROM DOCTOR D LEFT JOIN PATIENT P ON D.DoctorID = P.DoctorID
GROUP BY D.DoctorID, D.DoctorName;

SELECT D.DoctorID, D.DoctorName
FROM DOCTOR D LEFT JOIN PATIENT P ON D.DoctorID = P.DoctorID
WHERE P.PatientID IS NULL;
```

Expected results / notes

• The last query returns doctors who have no assigned patients.

Viva questions

Why use LEFT JOIN? It retains every doctor even when there is no matching patient.

What is `IS NULL` used for? It tests whether a value is NULL.

Question 6(B) — Explicit Cursor Equivalent in MySQL

Environment: MySQL Command Line

The source exercise calls for a PL/SQL explicit cursor. This is a MySQL stored-procedure equivalent that fetches employees with salary above 40000 into a temporary result table.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS show_high_salary_employees;
DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2)
);
INSERT INTO EMPLOYEE VALUES
(1,'Amit',60000),(2,'Priya',35000),(3,'Rahul',70000),(4,'Sneha',45000);

DROP TEMPORARY TABLE IF EXISTS CURSOR_RESULT;
CREATE TEMPORARY TABLE CURSOR_RESULT (EmpID INT, EmpName VARCHAR(50), Salary DECIMAL(10,2));

DELIMITER //
CREATE PROCEDURE show_high_salary_employees()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_id INT;
    DECLARE v_name VARCHAR(50);
    DECLARE v_salary DECIMAL(10,2);
    DECLARE cur CURSOR FOR SELECT EmpID, EmpName, Salary FROM EMPLOYEE WHERE Salary > 40000;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_id, v_name, v_salary;
        IF done = 1 THEN LEAVE read_loop; END IF;
        INSERT INTO CURSOR_RESULT VALUES (v_id, v_name, v_salary);
    END LOOP;
    CLOSE cur;
```

```
    SELECT * FROM CURSOR_RESULT;
    SELECT COUNT(*) AS EmployeeCount FROM CURSOR_RESULT;
END //
DELIMITER ;

CALL show_high_salary_employees();
```

Expected results / notes

• This is MySQL cursor syntax, not Oracle PL/SQL syntax.

Viva questions

What is an explicit cursor? A cursor explicitly declared by the programmer to process query results.

What does the NOT FOUND handler do? It marks the end of the cursor result set.

Question 7(A) — Account Table Queries

Environment: MySQL Command Line

Create an account table, select balances, calculate branch totals and average balance, show top three accounts, then update and delete records.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS ACCOUNT;
CREATE TABLE ACCOUNT (
    AccountNo INT PRIMARY KEY,
    HolderName VARCHAR(50),
    Branch VARCHAR(40),
    Balance DECIMAL(10,2)
);
INSERT INTO ACCOUNT VALUES
(101,'Amit','Pune',70000),(102,'Priya','Mumbai',45000),
(103,'Rahul','Pune',90000),(104,'Sneha','Delhi',55000),
(105,'Vijay','Mumbai',65000),(106,'Neha','Pune',40000),
(107,'Karan','Delhi',80000);

SELECT * FROM ACCOUNT WHERE Balance > 50000;
SELECT Branch, SUM(Balance) AS BranchTotal FROM ACCOUNT GROUP BY Branch;
SELECT AVG(Balance) AS AverageBalance FROM ACCOUNT;
SELECT * FROM ACCOUNT ORDER BY Balance DESC LIMIT 3;
UPDATE ACCOUNT SET Balance = Balance + 5000 WHERE AccountNo = 101;
DELETE FROM ACCOUNT WHERE AccountNo = 106;
SELECT * FROM ACCOUNT;
```

Expected results / notes

• `LIMIT 3` returns the first three rows after sorting.

Viva questions

What does SUM do? Adds numeric values.

What does LIMIT do? Restricts the number of returned rows.

Question 7(B) — Parameterized Department Cursor Equivalent

Environment: MySQL Command Line

This MySQL procedure accepts a department ID and displays employees in that department using a cursor. It is an equivalent exercise, not Oracle parameterized cursor syntax.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS show_department_employees;
DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
```

```
    DeptID INT,
    Salary DECIMAL(10,2)
);
INSERT INTO EMPLOYEE VALUES
(1,'Amit',10,60000),(2,'Priya',20,45000),(3,'Rahul',10,70000),
(4,'Sneha',20,50000),(5,'Vijay',30,55000);

DROP TEMPORARY TABLE IF EXISTS DEPT_CURSOR_RESULT;
CREATE TEMPORARY TABLE DEPT_CURSOR_RESULT
(EmpID INT, EmpName VARCHAR(50), DeptID INT, Salary DECIMAL(10,2));

DELIMITER //
CREATE PROCEDURE show_department_employees(IN p_DeptID INT)
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_id INT;
    DECLARE v_name VARCHAR(50);
    DECLARE v_dept INT;
    DECLARE v_salary DECIMAL(10,2);
    DECLARE cur CURSOR FOR
        SELECT EmpID, EmpName, DeptID, Salary FROM EMPLOYEE WHERE DeptID = p_DeptID;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    DELETE FROM DEPT_CURSOR_RESULT;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_id, v_name, v_dept, v_salary;
        IF done = 1 THEN LEAVE read_loop; END IF;
        INSERT INTO DEPT_CURSOR_RESULT VALUES (v_id, v_name, v_dept, v_salary);
    END LOOP;
    CLOSE cur;
    SELECT * FROM DEPT_CURSOR_RESULT;
END //
DELIMITER ;

CALL show_department_employees(10);
CALL show_department_employees(20);
```

Expected results / notes

• Call the procedure with a department ID to view matching employees.

Viva questions

What is a procedure parameter? A value passed to a stored procedure.

What is the purpose of a cursor? To process query results row by row.

Question 8(A) — Customer and Orders

Environment: MySQL Command Line

Create customer and orders tables; demonstrate joins, customers without orders, total order amount, and customers whose totals exceed 10000.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS ORDERS;
DROP TABLE IF EXISTS CUSTOMER;
CREATE TABLE CUSTOMER (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);
CREATE TABLE ORDERS (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    Amount DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);
INSERT INTO CUSTOMER VALUES (1,'Amit'),(2,'Priya'),(3,'Rahul'),(4,'Sneha');
INSERT INTO ORDERS VALUES (101,1,5000),(102,1,7000),(103,2,3000),(104,3,9000);

SELECT C.CustomerName, O.OrderID, O.Amount
FROM CUSTOMER C JOIN ORDERS O ON C.CustomerID = O.CustomerID;

SELECT C.CustomerID, C.CustomerName
FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CustomerID = O.CustomerID
```

```
WHERE O.OrderID IS NULL;

SELECT C.CustomerName, SUM(O.Amount) AS TotalAmount
FROM CUSTOMER C JOIN ORDERS O ON C.CustomerID = O.CustomerID
GROUP BY C.CustomerID, C.CustomerName;

SELECT C.CustomerName, SUM(O.Amount) AS TotalAmount
FROM CUSTOMER C JOIN ORDERS O ON C.CustomerID = O.CustomerID
GROUP BY C.CustomerID, C.CustomerName
HAVING SUM(O.Amount) > 10000;
```

Expected results / notes

• `HAVING` filters grouped customer totals.

Viva questions

What is the purpose of GROUP BY? Groups rows for aggregate calculations.

What is HAVING? A condition used to filter groups after aggregation.

Question 8(B) — SQL%ROWCOUNT Equivalent

Environment: MySQL Command Line

Oracle's SQL%ROWCOUNT is not available in MySQL. This procedure uses ROW_COUNT() to report how many employee rows were affected by a salary update.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS increase_department_salary;
DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2)
);
INSERT INTO EMPLOYEE VALUES
(1,'Amit',10,50000),(2,'Priya',20,45000),(3,'Rahul',10,60000);

DELIMITER //
CREATE PROCEDURE increase_department_salary(IN p_DeptID INT)
BEGIN
    UPDATE EMPLOYEE
    SET Salary = Salary * 1.10
    WHERE DeptID = p_DeptID;

    SELECT ROW_COUNT() AS RowsAffected;
END //
DELIMITER ;

CALL increase_department_salary(10);
SELECT * FROM EMPLOYEE;
```

Expected results / notes

• Employees in the selected department receive a 10% salary increase.

Viva questions

What does ROW_COUNT() return? The number of rows affected by the preceding statement.

What does `IN` mean for a procedure parameter? The caller passes a value into the procedure.

Question 9(A) — Employee Subqueries

Environment: MySQL Command Line

Use IN, an average-salary subquery, EXISTS, and NOT EXISTS.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;
```

```
DROP TABLE IF EXISTS EMPLOYEE;
DROP TABLE IF EXISTS DEPARTMENT;
CREATE TABLE DEPARTMENT (DeptID INT PRIMARY KEY, DeptName VARCHAR(40));
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    DeptID INT
);
INSERT INTO DEPARTMENT VALUES (10,'IT'),(20,'HR'),(30,'Finance');
INSERT INTO EMPLOYEE VALUES
(1,'Amit',60000,10),(2,'Priya',45000,20),(3,'Rahul',75000,10),(4,'Sneha',52000,30);

SELECT * FROM EMPLOYEE
WHERE DeptID IN (SELECT DeptID FROM DEPARTMENT WHERE DeptName IN ('IT','HR'));

SELECT * FROM EMPLOYEE
WHERE Salary > (SELECT AVG(Salary) FROM EMPLOYEE);

SELECT E.* FROM EMPLOYEE E
WHERE EXISTS (SELECT 1 FROM DEPARTMENT D WHERE D.DeptID = E.DeptID);

SELECT E.* FROM EMPLOYEE E
WHERE NOT EXISTS (SELECT 1 FROM DEPARTMENT D WHERE D.DeptID = E.DeptID);
```

Expected results / notes

• The last query returns employees whose department ID has no matching department.

Viva questions

What is IN? Checks whether a value matches any value in a set.

What is EXISTS? Tests whether a subquery returns at least one row.

Question 9(B) — Procedure with IN and OUT Parameters

Environment: MySQL Command Line

Create a procedure that counts employees in a specified department and returns the count through an OUT parameter.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS COUNT_DEPT_EMPLOYEES;
DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT
);
INSERT INTO EMPLOYEE VALUES (1,'Amit',10),(2,'Priya',20),(3,'Rahul',10),(4,'Sneha',10);

DELIMITER //
CREATE PROCEDURE COUNT_DEPT_EMPLOYEES(
    IN p_DeptID INT,
    OUT p_Count INT
)
BEGIN
    SELECT COUNT(*) INTO p_Count
    FROM EMPLOYEE
    WHERE DeptID = p_DeptID;
END //
DELIMITER ;

CALL COUNT_DEPT_EMPLOYEES(10, @total);
SELECT @total AS EmployeeCount;
```

Expected results / notes

• The OUT parameter returns the count for department 10.

Viva questions

What is an OUT parameter? A procedure parameter used to return a value to the caller.

What does COUNT(*) do? Counts rows matching the query conditions.

Question 10(A) — Employee Self JOIN

Environment: MySQL Command Line

Use a self join to display employees with managers, employees without managers, and employees earning more than their managers.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    ManagerID INT
);
INSERT INTO EMPLOYEE VALUES
(1,'Amit',80000,NULL),
(2,'Priya',50000,1),
(3,'Rahul',90000,1),
(4,'Sneha',60000,2),
(5,'Vijay',45000,2);

-- Employees and their managers
SELECT E.EmpName AS Employee, M.EmpName AS Manager
FROM EMPLOYEE E LEFT JOIN EMPLOYEE M ON E.ManagerID = M.EmpID;

-- Employees with no manager
SELECT EmpName FROM EMPLOYEE WHERE ManagerID IS NULL;

-- Employees earning more than their managers
SELECT E.EmpName AS Employee, E.Salary AS EmployeeSalary,
       M.EmpName AS Manager, M.Salary AS ManagerSalary
FROM EMPLOYEE E JOIN EMPLOYEE M ON E.ManagerID = M.EmpID
WHERE E.Salary > M.Salary;
```

Expected results / notes

• A self join joins a table to itself using aliases.

Viva questions

What is a self join? A join in which a table is joined to itself.

Why use aliases? To distinguish the two references to the same table.

Question 10(B) — Calculate Tax Function

Environment: MySQL Command Line

Create a MySQL function to calculate tax using the illustrative slabs used in the solution: up to 300000 at 0%, 300001–600000 at 5%, 600001–1000000 at 10%, and above 1000000 at 20%.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP FUNCTION IF EXISTS CALCULATE_TAX;
DELIMITER //
CREATE FUNCTION CALCULATE_TAX(income DECIMAL(12,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE tax DECIMAL(12,2) DEFAULT 0;

    IF income <= 300000 THEN
        SET tax = 0;
    ELSEIF income <= 600000 THEN
        SET tax = (income - 300000) * 0.05;
    ELSEIF income <= 1000000 THEN
        SET tax = 15000 + (income - 600000) * 0.10;
    ELSE
        SET tax = 55000 + (income - 1000000) * 0.20;
    END IF;
```

```
    RETURN tax;
END //
DELIMITER ;

SELECT CALCULATE_TAX(250000) AS Tax;
SELECT CALCULATE_TAX(500000) AS Tax;
SELECT CALCULATE_TAX(800000) AS Tax;
SELECT CALCULATE_TAX(1200000) AS Tax;
```

Expected results / notes

• These are illustrative slabs for the practical exercise, not tax advice or a statement of current tax law.

Viva questions

What is a function? A stored program that returns a value.

What does DETERMINISTIC mean? It declares that the same inputs produce the same result.

Question 11(A) — DDL and Constraints

Environment: MySQL Command Line

Demonstrate primary key, NOT NULL, UNIQUE, CHECK, DEFAULT, ALTER, rename, truncate, and drop. CHECK enforcement requires MySQL 8.0.16 or later.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS PRODUCT;
CREATE TABLE PRODUCT (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    SKU VARCHAR(30) UNIQUE,
    Price DECIMAL(10,2) CHECK (Price >= 0),
    Stock INT DEFAULT 0
);

INSERT INTO PRODUCT VALUES (1,'Laptop','SKU101',55000,10);
INSERT INTO PRODUCT (ProductID, ProductName, SKU, Price)
VALUES (2,'Mouse','SKU102',800);

ALTER TABLE PRODUCT ADD COLUMN Category VARCHAR(40);
ALTER TABLE PRODUCT MODIFY ProductName VARCHAR(80) NOT NULL;
ALTER TABLE PRODUCT RENAME TO PRODUCT_RENAMED;

SELECT * FROM PRODUCT_RENAMED;

-- Use these destructive commands only on the practice table:
TRUNCATE TABLE PRODUCT_RENAMED;
DROP TABLE PRODUCT_RENAMED;
```

Expected results / notes

• `TRUNCATE` removes all rows while retaining the table; `DROP` removes the table definition and its data.

Viva questions

What is a constraint? A rule that restricts values allowed in a table.

What is the difference between TRUNCATE and DROP? TRUNCATE removes rows; DROP removes the table itself.

Question 11(B) — Update Salary Procedure

Environment: MySQL Command Line

Create a procedure that updates an employee's salary by a percentage, validates the employee ID and non-negative percentage, and displays the result.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP PROCEDURE IF EXISTS UPDATE_SALARY;
```

```
DROP TABLE IF EXISTS EMPLOYEE;
CREATE TABLE EMPLOYEE (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2)
);
INSERT INTO EMPLOYEE VALUES (1,'Amit',50000),(2,'Priya',45000);

DELIMITER //
CREATE PROCEDURE UPDATE_SALARY(IN p_EmpID INT, IN p_Percentage DECIMAL(5,2))
BEGIN
    IF p_Percentage < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Percentage cannot be negative';
    ELSEIF NOT EXISTS (SELECT 1 FROM EMPLOYEE WHERE EmpID = p_EmpID) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Employee ID not found';
    ELSE
        UPDATE EMPLOYEE
        SET Salary = Salary + (Salary * p_Percentage / 100)
        WHERE EmpID = p_EmpID;
        SELECT * FROM EMPLOYEE WHERE EmpID = p_EmpID;
    END IF;
END //
DELIMITER ;

CALL UPDATE_SALARY(1, 10);
```

Expected results / notes

• The example increases employee 1's salary by 10%.

Viva questions

What does SIGNAL do? Raises a SQL condition/error from a stored program.

What does percentage-based salary update mean? Adds a fraction of the existing salary to the salary.

Question 12(A) — UNION, UNION ALL, Intersection, and Minus

Environment: MySQL Command Line

Compare two student tables using UNION, UNION ALL, an intersection equivalent, and a minus equivalent.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS STUDENT_A;
DROP TABLE IF EXISTS STUDENT_B;
CREATE TABLE STUDENT_A (RollNo INT PRIMARY KEY, Name VARCHAR(50));
CREATE TABLE STUDENT_B (RollNo INT PRIMARY KEY, Name VARCHAR(50));

INSERT INTO STUDENT_A VALUES (1,'Amit'),(2,'Priya'),(3,'Rahul');
INSERT INTO STUDENT_B VALUES (2,'Priya'),(3,'Rahul'),(4,'Sneha');

-- UNION removes duplicates
SELECT RollNo, Name FROM STUDENT_A
UNION
SELECT RollNo, Name FROM STUDENT_B;

-- UNION ALL retains duplicates
SELECT RollNo, Name FROM STUDENT_A
UNION ALL
SELECT RollNo, Name FROM STUDENT_B;

-- Intersection equivalent
SELECT A.RollNo, A.Name
FROM STUDENT_A A INNER JOIN STUDENT_B B ON A.RollNo = B.RollNo;

-- Minus equivalent
SELECT A.RollNo, A.Name
FROM STUDENT_A A
WHERE NOT EXISTS (
    SELECT 1 FROM STUDENT_B B WHERE B.RollNo = A.RollNo
);
```

Expected results / notes

• MySQL does not provide Oracle's MINUS operator; NOT EXISTS is used as an equivalent.

Viva questions

UNION vs UNION ALL? UNION removes duplicate rows; UNION ALL retains them.

What is an intersection? Rows common to both query results.

Question 12(B) — MongoDB Delete Operations

Environment: MongoDB Shell (mongosh)

Demonstrate deleteOne(), deleteMany(), and dropping a demonstration collection.

Complete code

```
use DBMS_Practical

db.products.drop()
db.products.insertMany([
  { productId: 1, name: "Laptop", category: "Electronics" },
  { productId: 2, name: "Mouse", category: "Electronics" },
  { productId: 3, name: "Chair", category: "Furniture" },
  { productId: 4, name: "Desk", category: "Furniture" },
  { productId: 5, name: "Keyboard", category: "Electronics" }
])

// Delete one matching document
db.products.deleteOne({ productId: 1 })

// Delete all documents in the Furniture category
db.products.deleteMany({ category: "Furniture" })

db.products.find()

// Drop the entire demonstration collection
db.products.drop()
```

Expected results / notes

• `drop()` removes the collection and its documents.

Viva questions

What is deleteOne()? Deletes the first document matching a filter.

What is deleteMany()? Deletes all documents matching a filter.

Question 13(A) — Sales Aggregation Queries

Environment: MySQL Command Line

Create a sales table and calculate total sales, category revenue, product quantities, monthly sales, and categories with revenue above 20000.

Complete code

```
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS SALES;
CREATE TABLE SALES (
    SaleID INT PRIMARY KEY,
    Product VARCHAR(50),
    Category VARCHAR(40),
    Quantity INT,
    UnitPrice DECIMAL(10,2),
    SaleDate DATE
);

INSERT INTO SALES VALUES
(1,'Laptop','Electronics',2,50000,'2026-01-05'),
(2,'Mouse','Electronics',10,500,'2026-01-10'),
(3,'Chair','Furniture',5,3000,'2026-02-03'),
(4,'Desk','Furniture',2,7000,'2026-02-15'),
(5,'Keyboard','Electronics',4,1500,'2026-03-08');

SELECT SUM(Quantity * UnitPrice) AS TotalSales FROM SALES;

SELECT Category, SUM(Quantity * UnitPrice) AS CategoryRevenue
```

```
FROM SALES GROUP BY Category;

SELECT Product, SUM(Quantity) AS TotalQuantity
FROM SALES GROUP BY Product;

SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS SalesMonth,
       SUM(Quantity * UnitPrice) AS MonthlySales
FROM SALES
GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
ORDER BY SalesMonth;

SELECT Category, SUM(Quantity * UnitPrice) AS Revenue
FROM SALES
GROUP BY Category
HAVING SUM(Quantity * UnitPrice) > 20000;
```

Expected results / notes

• Revenue is calculated as quantity multiplied by unit price.

Viva questions

What does HAVING do? Filters grouped aggregate results.

What does DATE_FORMAT do? Formats a date value as text using the specified pattern.

Question 13(B) — MongoDB Employee Aggregation

Environment: MongoDB Shell (mongosh)

Filter employees with salary above 30000, project name and salary, and sort salary in descending order.

Complete code

```
use DBMS_Practical

db.employees.drop()
db.employees.insertMany([
  { empId: 1, name: "Amit", department: "IT", salary: 60000 },
  { empId: 2, name: "Priya", department: "HR", salary: 25000 },
  { empId: 3, name: "Rahul", department: "IT", salary: 70000 },
  { empId: 4, name: "Sneha", department: "Finance", salary: 45000 }
])

db.employees.aggregate([
  { $match: { salary: { $gt: 30000 } } },
  { $project: { _id: 0, name: 1, salary: 1 } },
  { $sort: { salary: -1 } }
])
```

Expected results / notes

• `$match` filters, `$project` chooses fields, and `$sort` orders the output.

Viva questions

What does `$match` do? Filters documents by conditions.

What does `$project` do? Specifies which fields appear in output documents.

Question 14(A) — Student Result Analysis Using Views

Environment: MySQL Command Line

Create a student marks table, create a view showing marks at least 60, retrieve view records, update underlying data, and drop the view.

Complete code

```
-- QUESTION 14(A): STUDENT RESULT ANALYSIS USING VIEWS

-- STEP 1: Create and select database
CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

-- STEP 2: Remove old view and table
DROP VIEW IF EXISTS STUDENT_RESULT_VIEW;
DROP TABLE IF EXISTS STUDENT_MARKS;
```

```
-- STEP 3: Create student marks table
CREATE TABLE STUDENT_MARKS (
    RollNo INT,
    Name VARCHAR(50) NOT NULL,
    Subject VARCHAR(50),
    Marks INT CHECK (Marks BETWEEN 0 AND 100),
    PRIMARY KEY (RollNo, Subject)
);

-- STEP 4: Insert sample records
INSERT INTO STUDENT_MARKS VALUES
(1, 'Amit', 'DBMS', 85),
(1, 'Amit', 'Java', 72),
(2, 'Priya', 'DBMS', 95),
(2, 'Priya', 'Java', 55),
(3, 'Rahul', 'DBMS', 48),
(3, 'Rahul', 'Java', 65),
(4, 'Sneha', 'DBMS', 60),
(4, 'Sneha', 'Java', 88);

-- STEP 5: Create view for marks >= 60
CREATE VIEW STUDENT_RESULT_VIEW AS
SELECT RollNo, Name, Subject, Marks
FROM STUDENT_MARKS
WHERE Marks >= 60;

-- STEP 6: Display records from the view
SELECT * FROM STUDENT_RESULT_VIEW;

-- STEP 7: Update underlying table
UPDATE STUDENT_MARKS
SET Marks = 75
WHERE RollNo = 3 AND Subject = 'DBMS';

-- STEP 8: Display view after update
SELECT * FROM STUDENT_RESULT_VIEW;

-- STEP 9: Drop the view
DROP VIEW STUDENT_RESULT_VIEW;

-- STEP 10: Verify underlying table still exists
SELECT * FROM STUDENT_MARKS;
```

Expected results / notes

• The view initially displays six rows. After Rahul's DBMS marks change from 48 to 75, the view displays seven rows.

Viva questions

What is a view? A virtual table defined by a SELECT query.

What happens when a view is dropped? The view definition is removed, but the underlying table remains.

Question 14(B) — Hospital Patient Records

Environment: MongoDB Shell (mongosh)

Create a patients collection, insert five documents, retrieve patients older than 60, filter by disease, and count patients by disease.

Complete code

```
// QUESTION 14(B): HOSPITAL PATIENT RECORDS

// STEP 1: Select database
use DBMS_Practical

// STEP 2: Remove old collection for a clean run
db.patients.drop()

// STEP 3: Insert at least 5 patient documents
db.patients.insertMany([
  { patientId: 101, name: "Amit", age: 65, disease: "Diabetes", doctor: "Dr. Sharma" },
  { patientId: 102, name: "Priya", age: 45, disease: "Fever", doctor: "Dr. Patil" },
  { patientId: 103, name: "Rahul", age: 72, disease: "Diabetes", doctor: "Dr. Sharma" },
  { patientId: 104, name: "Sneha", age: 30, disease: "Asthma", doctor: "Dr. Joshi" },
  { patientId: 105, name: "Vijay", age: 61, disease: "Fever", doctor: "Dr. Patil" }
])

// STEP 4: Display all patients
```

```
db.patients.find()

// STEP 5: Find patients older than 60
db.patients.find({ age: { $gt: 60 } })

// STEP 6: Find patients with Diabetes
db.patients.find({ disease: "Diabetes" })

// STEP 7: Count patients for each disease
db.patients.aggregate([
  {
    $group: {
      _id: "$disease",
      totalPatients: { $sum: 1 }
    }
  },
  {
    $sort: { _id: 1 }
  }
])
```

Expected results / notes

• Disease counts: Asthma 1, Diabetes 2, Fever 2.

Viva questions

What does `$gt` mean? Greater than.

What does `$sum: 1` do? Counts documents in each group.

Question 15(A) — Employee Database using JOINs, Subqueries, and Views

Environment: MySQL Command Line

Create Employee, Department, and Project tables; demonstrate JOINs, IN and EXISTS subqueries, and a view.

Complete code

```
-- QUESTION 15(A): EMPLOYEE DATABASE USING JOINS,
-- SUBQUERIES AND VIEWS

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP VIEW IF EXISTS Employee_Department_View;
DROP TABLE IF EXISTS Project;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Department;

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL
);

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(50) NOT NULL,
    EmpID INT,
    FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)
);

INSERT INTO Department VALUES
(10, 'IT'), (20, 'HR'), (30, 'Finance'), (40, 'Marketing');

INSERT INTO Employee VALUES
(1, 'Amit', 60000, 10),
(2, 'Priya', 55000, 20),
(3, 'Rahul', 70000, 10),
(4, 'Sneha', 45000, 30),
(5, 'Vijay', 50000, NULL);
```

```
INSERT INTO Project VALUES
(101, 'Website Development', 1),
(102, 'Payroll System', 2),
(103, 'Database Migration', 3),
(104, 'Financial Analysis', 4);

-- INNER JOIN - Employees with departments
SELECT E.EmpID, E.EmpName, D.DeptName, E.Salary
FROM Employee E
INNER JOIN Department D ON E.DeptID = D.DeptID;

-- LEFT JOIN - All employees, including employees without a department
SELECT E.EmpName, D.DeptName
FROM Employee E LEFT JOIN Department D ON E.DeptID = D.DeptID;

-- RIGHT JOIN - All departments, including departments without employees
SELECT E.EmpName, D.DeptName
FROM Employee E RIGHT JOIN Department D ON E.DeptID = D.DeptID;

-- Find employees in IT or HR using IN
SELECT EmpID, EmpName, Salary
FROM Employee
WHERE DeptID IN (
    SELECT DeptID FROM Department WHERE DeptName IN ('IT', 'HR')
);

-- Find employees assigned to projects using EXISTS
SELECT E.EmpID, E.EmpName
FROM Employee E
WHERE EXISTS (
    SELECT 1 FROM Project P WHERE P.EmpID = E.EmpID
);

-- Create a view
CREATE VIEW Employee_Department_View AS
SELECT E.EmpName, D.DeptName, E.Salary
FROM Employee E INNER JOIN Department D ON E.DeptID = D.DeptID;

-- Display the view
SELECT * FROM Employee_Department_View;

-- Optional: Drop the view
DROP VIEW Employee_Department_View;
```

Expected results / notes

• The LEFT JOIN includes Vijay with a NULL department; the RIGHT JOIN includes Marketing even though no employee belongs to it.

Viva questions

What is a JOIN? Combines rows from related tables.

What is a view? A virtual table defined by a SELECT statement.

Question 15(B) — MongoDB Array Fields and Array Queries

Environment: MongoDB Shell (mongosh)

Find students with specified skills and update skill arrays using $push and $addToSet.

Complete code

```
// QUESTION 15(B): ARRAY FIELDS AND ARRAY QUERIES

use DBMS_Practical
db.students.drop()

db.students.insertMany([
  { rollNo: 1, name: "Amit", skills: ["Java", "Python", "SQL"] },
  { rollNo: 2, name: "Priya", skills: ["Python", "MongoDB"] },
  { rollNo: 3, name: "Rahul", skills: ["C++", "JavaScript"] },
  { rollNo: 4, name: "Sneha", skills: ["Java", "MongoDB", "SQL"] },
  { rollNo: 5, name: "Vijay", skills: ["Python", "C++"] }
])

db.students.find()
db.students.find({ skills: "Java" })
db.students.find({ skills: { $in: ["Python", "MongoDB"] } })

db.students.updateOne(
```

```
  { rollNo: 1 },
  { $push: { skills: "HTML" } }
)
db.students.find({ rollNo: 1 })

db.students.updateOne(
  { rollNo: 2 },
  { $addToSet: { skills: "Java" } }
)
db.students.updateOne(
  { rollNo: 2 },
  { $addToSet: { skills: "Java" } }
)

db.students.find()
```

Expected results / notes

• `$push` appends a value; `$addToSet` adds a value only if it is not already present.

Viva questions

What is an array in MongoDB? A field that stores multiple values.

What is the difference between $push and $addToSet? $push can add duplicates; $addToSet avoids duplicate values.

Question 16(A) — University Examination Database

Environment: MySQL Command Line

Create STUDENT and RESULT tables with primary and foreign keys, then calculate totals, averages, subject-wise highest marks, failed students, and averages above the overall average.

Complete code

```
-- QUESTION 16(A): UNIVERSITY EXAMINATION DATABASE

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS RESULT;
DROP TABLE IF EXISTS STUDENT;

CREATE TABLE STUDENT (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Department VARCHAR(30)
);

CREATE TABLE RESULT (
    RollNo INT,
    Subject VARCHAR(50),
    Marks INT,
    PRIMARY KEY (RollNo, Subject),
    FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo)
);

INSERT INTO STUDENT VALUES
(1, 'Amit', 'IT'), (2, 'Priya', 'CS'), (3, 'Rahul', 'IT'),
(4, 'Sneha', 'CS'), (5, 'Vijay', 'IT');

INSERT INTO RESULT VALUES
(1, 'DBMS', 80), (1, 'Java', 70), (1, 'CN', 65),
(2, 'DBMS', 90), (2, 'Java', 85), (2, 'CN', 78),
(3, 'DBMS', 35), (3, 'Java', 55), (3, 'CN', 60),
(4, 'DBMS', 75), (4, 'Java', 80), (4, 'CN', 85),
(5, 'DBMS', 40), (5, 'Java', 45), (5, 'CN', 50);

-- Student-wise total marks
SELECT S.RollNo, S.Name, SUM(R.Marks) AS TotalMarks
FROM STUDENT S JOIN RESULT R ON S.RollNo = R.RollNo
GROUP BY S.RollNo, S.Name;

-- Student-wise average marks
SELECT S.RollNo, S.Name, AVG(R.Marks) AS AverageMarks
FROM STUDENT S JOIN RESULT R ON S.RollNo = R.RollNo
GROUP BY S.RollNo, S.Name;

-- Highest marks in each subject
SELECT Subject, MAX(Marks) AS HighestMarks
```

```
FROM RESULT GROUP BY Subject;

-- Students who failed in any subject (below 40)
SELECT DISTINCT S.RollNo, S.Name
FROM STUDENT S JOIN RESULT R ON S.RollNo = R.RollNo
WHERE R.Marks < 40;

-- Students whose average exceeds the overall average
SELECT S.RollNo, S.Name, AVG(R.Marks) AS AverageMarks
FROM STUDENT S JOIN RESULT R ON S.RollNo = R.RollNo
GROUP BY S.RollNo, S.Name
HAVING AVG(R.Marks) > (SELECT AVG(Marks) FROM RESULT);
```

Expected results / notes

• Assuming marks below 40 means failure, Rahul failed DBMS with 35 marks. Overall average is 66.2; Amit, Priya, and Sneha are above it.

Viva questions

What is a foreign key? A column referencing a key in another table.

What is HAVING? Filters groups after aggregation.

Question 16(B) — MongoDB Indexing

Environment: MongoDB Shell (mongosh)

Create single-field, compound, and unique indexes; query indexed fields; inspect indexes and query plans.

Complete code

```
// QUESTION 16(B): MONGODB INDEXING
use DBMS_Practical
db.products.drop()

db.products.insertMany([
  { productId: 101, name: "Laptop", category: "Electronics", price: 55000, stock: 10 },
  { productId: 102, name: "Mouse", category: "Electronics", price: 800, stock: 50 },
  { productId: 103, name: "Chair", category: "Furniture", price: 3500, stock: 15 },
  { productId: 104, name: "Desk", category: "Furniture", price: 7000, stock: 8 },
  { productId: 105, name: "Keyboard", category: "Electronics", price: 1500, stock: 25 }
])

db.products.createIndex({ category: 1 })
db.products.createIndex({ category: 1, price: -1 })
db.products.createIndex({ productId: 1 }, { unique: true })

db.products.find({ category: "Electronics" })
db.products.find({ category: "Electronics" }).sort({ price: -1 })
db.products.getIndexes()
db.products.find({ category: "Electronics" }).explain("executionStats")
```

Expected results / notes

• `1` is ascending; `-1` is descending. `explain()` shows the plan and execution statistics; the planner may choose an index scan when appropriate.

Viva questions

What is a compound index? An index on two or more fields.

What does explain() do? Shows how MongoDB plans to execute a query.

Question 17(A) — SQL Transactions

Environment: MySQL Command Line

Demonstrate COMMIT, ROLLBACK, and SAVEPOINT using a money transfer between two accounts. Execute in order in the same session.

Complete code

```
-- QUESTION 17(A): SQL TRANSACTIONS

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;
```

```
DROP TABLE IF EXISTS ACCOUNT;
CREATE TABLE ACCOUNT (
    AccountNo INT PRIMARY KEY,
    AccountHolder VARCHAR(50),
    Balance DECIMAL(10,2)
) ENGINE = InnoDB;

INSERT INTO ACCOUNT VALUES
(101, 'Amit', 10000.00),
(102, 'Priya', 5000.00);

SELECT * FROM ACCOUNT;

-- Transfer 1000 from Amit to Priya
START TRANSACTION;
UPDATE ACCOUNT SET Balance = Balance - 1000 WHERE AccountNo = 101;
UPDATE ACCOUNT SET Balance = Balance + 1000 WHERE AccountNo = 102;
SELECT * FROM ACCOUNT;
COMMIT;
SELECT * FROM ACCOUNT;

-- Demonstrate ROLLBACK
START TRANSACTION;
UPDATE ACCOUNT SET Balance = Balance - 500 WHERE AccountNo = 101;
UPDATE ACCOUNT SET Balance = Balance + 500 WHERE AccountNo = 102;
SELECT * FROM ACCOUNT;
ROLLBACK;
SELECT * FROM ACCOUNT;

-- Demonstrate SAVEPOINT
START TRANSACTION;
UPDATE ACCOUNT SET Balance = Balance - 200 WHERE AccountNo = 101;
SAVEPOINT after_debit;
UPDATE ACCOUNT SET Balance = Balance + 200 WHERE AccountNo = 102;
SELECT * FROM ACCOUNT;
ROLLBACK TO SAVEPOINT after_debit;
SELECT * FROM ACCOUNT;
ROLLBACK;
SELECT * FROM ACCOUNT;
```

Expected results / notes

• After the committed transfer, balances are Amit 9000 and Priya 6000. The later test changes are rolled back. Use InnoDB and the same session.

Viva questions

What does COMMIT do? Permanently saves transaction changes.

What does SAVEPOINT do? Marks a point in a transaction for partial rollback.

Question 17(B) — MongoDB Books Collection

Environment: MongoDB Shell (mongosh)

Insert five books, update one book, update all Programming books, and increment stock for every book.

Complete code

```
// QUESTION 17(B): MONGODB BOOKS COLLECTION
use DBMS_Practical
db.books.drop()

db.books.insertMany([
  { ISBN: "B101", title: "DBMS Basics", author: "Amit", category: "Education", price: 450, stock: 10 },
  { ISBN: "B102", title: "Java Programming", author: "Priya", category: "Programming", price: 600, stock:
 15 },
  { ISBN: "B103", title: "Python Guide", author: "Rahul", category: "Programming", price: 550, stock: 12
},
  { ISBN: "B104", title: "Web Development", author: "Sneha", category: "Programming", price: 700, stock:
8 },
  { ISBN: "B105", title: "Database Design", author: "Vijay", category: "Education", price: 500, stock:
20 }
])

db.books.find()

db.books.updateOne(
  { ISBN: "B101" },
  { $set: { price: 480 } }
```

```
)

db.books.updateMany(
  { category: "Programming" },
  { $set: { price: 650 } }
)

db.books.updateMany(
  {},
  { $inc: { stock: 5 } }
)

db.books.find()
```

Expected results / notes

• Final prices: B101 480; B102, B103, B104 650; B105 500. Stock increases by five for every book.

Viva questions

What does $inc do? Increases or decreases a numeric field.

What does an empty filter {} match? All documents in the collection.

Question 18(A) — Inventory Database using Nested Queries

Environment: MySQL Command Line

Find products above average price, most expensive products in each category, below-average stock, and categories with products priced above 1000.

Complete code

```
-- QUESTION 18(A): INVENTORY DATABASE USING NESTED QUERIES

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;
DROP TABLE IF EXISTS INVENTORY;

CREATE TABLE INVENTORY (
    ItemID INT PRIMARY KEY,
    ItemName VARCHAR(50) NOT NULL,
    Category VARCHAR(30),
    Price DECIMAL(10,2),
    Stock INT
);

INSERT INTO INVENTORY VALUES
(1, 'Laptop', 'Electronics', 55000, 10),
(2, 'Mouse', 'Electronics', 800, 50),
(3, 'Keyboard', 'Electronics', 1500, 25),
(4, 'Chair', 'Furniture', 3500, 15),
(5, 'Desk', 'Furniture', 7000, 8),
(6, 'Notebook', 'Stationery', 100, 100),
(7, 'Pen', 'Stationery', 20, 200),
(8, 'Monitor', 'Electronics', 12000, 12);

SELECT * FROM INVENTORY;

-- Products priced above average price
SELECT * FROM INVENTORY
WHERE Price > (SELECT AVG(Price) FROM INVENTORY);

-- Most expensive product in each category
SELECT * FROM INVENTORY I
WHERE Price = (
    SELECT MAX(Price) FROM INVENTORY WHERE Category = I.Category
);

-- Products with below-average stock
SELECT * FROM INVENTORY
WHERE Stock < (SELECT AVG(Stock) FROM INVENTORY);

-- Categories containing products costing more than 1000
SELECT DISTINCT Category FROM INVENTORY
WHERE Category IN (
    SELECT Category FROM INVENTORY WHERE Price > 1000
);
```

Expected results / notes

• Average price is 9997.50. The most expensive items by category are Laptop, Desk, and Notebook. Average stock is 52.5.

Viva questions

What is a nested query? A query inside another query.

What is a correlated subquery? A subquery that refers to a column from the outer query.

Question 18(B) — MongoDB Admissions Management

Environment: MongoDB Shell (mongosh)

Insert admission records, use comparison and logical filters, create indexes, aggregate course-wise counts and average scores, update a record, and delete one.

Complete code

```
// QUESTION 18(B): MONGODB ADMISSIONS MANAGEMENT
use DBMS_Practical
db.admissions.drop()

db.admissions.insertMany([
  { applicationNo: 1001, studentName: "Amit", course: "BTech IT", category: "OPEN", entranceScore: 85,
admissionStatus: "Admitted" },
  { applicationNo: 1002, studentName: "Priya", course: "BTech CS", category: "OBC", entranceScore: 92,
admissionStatus: "Admitted" },
  { applicationNo: 1003, studentName: "Rahul", course: "BTech IT", category: "SC", entranceScore: 65,
admissionStatus: "Pending" },
  { applicationNo: 1004, studentName: "Sneha", course: "BTech CS", category: "OPEN", entranceScore: 78,
admissionStatus: "Pending" },
  { applicationNo: 1005, studentName: "Vijay", course: "BTech IT", category: "OBC", entranceScore: 55,
admissionStatus: "Rejected" }
])

db.admissions.find()
db.admissions.find({ entranceScore: { $gt: 80 } })

db.admissions.find({
  $or: [
    { category: "OPEN" },
    { entranceScore: { $gt: 90 } }
  ]
})

db.admissions.find({
  $and: [
    { course: "BTech IT" },
    { entranceScore: { $gt: 60 } }
  ]
})

db.admissions.createIndex({ applicationNo: 1 }, { unique: true })
db.admissions.createIndex({ course: 1 })
db.admissions.getIndexes()

db.admissions.aggregate([
  {
    $group: {
      _id: "$course",
      totalApplications: { $sum: 1 },
      averageScore: { $avg: "$entranceScore" }
    }
  },
  { $sort: { _id: 1 } }
])

db.admissions.updateOne(
  { applicationNo: 1003 },
  { $set: { admissionStatus: "Admitted" } }
)
db.admissions.find({ applicationNo: 1003 })

db.admissions.deleteOne({ applicationNo: 1005 })
db.admissions.find()
```

Expected results / notes

• Before update and delete, BTech CS has 2 applications with average 85; BTech IT has 3 applications with average 68.33.

Viva questions

What is CRUD? Create, Read, Update, Delete.

What is a unique index? It prevents duplicate indexed values.

Question 19(A) — SQL Sequence and Index

Environment: MySQL Command Line

MySQL does not use Oracle-style CREATE SEQUENCE syntax in the same way. This solution uses AUTO_INCREMENT for IDs and an index on City.

Complete code

```
-- QUESTION 19(A): SQL SEQUENCE AND INDEX
-- MySQL-compatible implementation

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;
DROP TABLE IF EXISTS CUSTOMER;

CREATE TABLE CUSTOMER (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    City VARCHAR(40),
    Balance DECIMAL(10,2)
);

INSERT INTO CUSTOMER (CustomerName, City, Balance)
VALUES
('Amit', 'Pune', 5000),
('Priya', 'Mumbai', 8000),
('Rahul', 'Pune', 3500),
('Sneha', 'Delhi', 6000),
('Vijay', 'Mumbai', 4500);

SELECT * FROM CUSTOMER;

CREATE INDEX idx_city ON CUSTOMER(City);

SELECT * FROM CUSTOMER WHERE City = 'Pune';

SELECT CustomerID, CustomerName FROM CUSTOMER;

SELECT AUTO_INCREMENT
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'DBMS_Practical'
AND TABLE_NAME = 'CUSTOMER';

SHOW INDEX FROM CUSTOMER;
DROP INDEX idx_city ON CUSTOMER;
SHOW INDEX FROM CUSTOMER;
```

Expected results / notes

• The generated IDs normally run from 1 to 5; the next AUTO_INCREMENT value is normally 6 after these inserts.

Viva questions

What is AUTO_INCREMENT? A MySQL feature that generates numeric column values automatically.

What does DROP INDEX do? Removes an index from a table.

Question 19(B) — MongoDB Employee Aggregation

Environment: MongoDB Shell (mongosh)

Group employees by department, calculate employee count and average salary, and sort by average salary ascending.

Complete code

```
// QUESTION 19(B): EMPLOYEE AGGREGATION
use DBMS_Practical
db.employees.drop()

db.employees.insertMany([
  { empId: 101, name: "Amit", department: "IT", salary: 60000 },
```

```
  { empId: 102, name: "Priya", department: "HR", salary: 40000 },
  { empId: 103, name: "Rahul", department: "IT", salary: 70000 },
  { empId: 104, name: "Sneha", department: "HR", salary: 50000 },
  { empId: 105, name: "Vijay", department: "Finance", salary: 55000 },
  { empId: 106, name: "Neha", department: "Finance", salary: 65000 }
])

db.employees.find()

db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      totalEmployees: { $sum: 1 },
      averageSalary: { $avg: "$salary" }
    }
  },
  { $sort: { averageSalary: 1 } }
])
```

Expected results / notes

• Expected order: HR average 45000, Finance average 60000, IT average 65000.

Viva questions

What does $group do? Groups documents by a field.

What does $sort: {averageSalary: 1} do? Sorts by average salary ascending.

Question 20(A) — Attendance Management System

Environment: MySQL Command Line

Calculate attendance percentages, identify students below 75%, compute subject-wise average attendance, and find students meeting 75% in every subject.

Complete code

```
-- QUESTION 20(A): ATTENDANCE MANAGEMENT SYSTEM

CREATE DATABASE IF NOT EXISTS DBMS_Practical;
USE DBMS_Practical;

DROP TABLE IF EXISTS ATTENDANCE;
DROP TABLE IF EXISTS STUDENT;

CREATE TABLE STUDENT (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Department VARCHAR(30)
);

CREATE TABLE ATTENDANCE (
    RollNo INT,
    Subject VARCHAR(50),
    TotalClasses INT,
    AttendedClasses INT,
    PRIMARY KEY (RollNo, Subject),
    FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo)
);

INSERT INTO STUDENT VALUES
(1, 'Amit', 'IT'), (2, 'Priya', 'IT'), (3, 'Rahul', 'CS'),
(4, 'Sneha', 'CS'), (5, 'Vijay', 'IT');

INSERT INTO ATTENDANCE VALUES
(1, 'DBMS', 40, 35), (1, 'Java', 40, 32),
(2, 'DBMS', 40, 28), (2, 'Java', 40, 30),
(3, 'DBMS', 40, 20), (3, 'Java', 40, 25),
(4, 'DBMS', 40, 36), (4, 'Java', 40, 38),
(5, 'DBMS', 40, 30), (5, 'Java', 40, 29);

-- Calculate attendance percentage
SELECT S.RollNo, S.Name, A.Subject, A.TotalClasses, A.AttendedClasses,
       ROUND(A.AttendedClasses * 100.0 / A.TotalClasses, 2) AS AttendancePercentage
FROM STUDENT S JOIN ATTENDANCE A ON S.RollNo = A.RollNo;

-- Students below 75% in any subject
SELECT DISTINCT S.RollNo, S.Name
```

```
FROM STUDENT S JOIN ATTENDANCE A ON S.RollNo = A.RollNo
WHERE A.AttendedClasses * 100.0 / A.TotalClasses < 75;

-- Subject-wise average attendance
SELECT Subject,
       ROUND(AVG(AttendedClasses * 100.0 / TotalClasses), 2) AS AverageAttendancePercentage
FROM ATTENDANCE GROUP BY Subject;

-- Students meeting 75% in every subject
SELECT S.RollNo, S.Name
FROM STUDENT S JOIN ATTENDANCE A ON S.RollNo = A.RollNo
GROUP BY S.RollNo, S.Name
HAVING MIN(A.AttendedClasses * 100.0 / A.TotalClasses) >= 75;
```

Expected results / notes

• Below 75% in at least one subject: Priya, Rahul, Vijay. Subject averages: DBMS 74.50%; Java 77.00%. Students meeting 75% in every subject: Amit and Sneha.

Viva questions

How is attendance percentage calculated? AttendedClasses / TotalClasses × 100.

Why use HAVING MIN(...) >= 75? It ensures the student's lowest subject attendance is at least 75%.

Question 20(B) — MongoDB Sales Analysis

Environment: MongoDB Shell (mongosh)

Group sales by category and calculate total quantity and revenue; sort categories by revenue descending.

Complete code

```
// QUESTION 20(B): MONGODB SALES ANALYSIS
use DBMS_Practical
db.sales.drop()

db.sales.insertMany([
  { product: "Laptop", category: "Electronics", quantity: 2, unitPrice: 50000, date: ISODate("2026-10-
01") },
  { product: "Mouse", category: "Electronics", quantity: 10, unitPrice: 500, date: ISODate("2026-10-02")
},
  { product: "Chair", category: "Furniture", quantity: 5, unitPrice: 3000, date: ISODate("2026-10-03") },
  { product: "Desk", category: "Furniture", quantity: 2, unitPrice: 7000, date: ISODate("2026-10-04") },
  { product: "Keyboard", category: "Electronics", quantity: 4, unitPrice: 1500, date: ISODate("2026-10-
05") },
  { product: "Notebook", category: "Stationery", quantity: 20, unitPrice: 50, date: ISODate("2026-10-
06") }
])

db.sales.find()

db.sales.aggregate([
  {
    $group: {
      _id: "$category",
      totalQuantity: { $sum: "$quantity" },
      totalRevenue: {
        $sum: { $multiply: ["$quantity", "$unitPrice"] }
      }
    }
  },
  { $sort: { totalRevenue: -1 } }
])
```

Expected results / notes

• Expected results: Electronics quantity 16 and revenue 111000; Furniture quantity 7 and revenue 29000; Stationery quantity 20 and revenue 1000.

Viva questions

What does $multiply do? Multiplies the specified values.

What does $sort with -1 do? Sorts in descending order.

Final Revision Notes

• Run MySQL solutions in MySQL Command Line. Each SQL script starts by creating/selecting DBMS_Practical.

• Run MongoDB solutions in mongosh, not the MySQL client.

• Oracle PL/SQL topics in this document use MySQL equivalents where described; they are not identical to Oracle PL/SQL.

• Destructive statements such as DROP TABLE, DROP VIEW, collection.drop(), DELETE, and TRUNCATE are intended for the practice data in this document.

• Transaction examples should be executed in order in the same MySQL session.